



PostgreSQL Database Fundamentals

Eng. Fatma Elraey

Day 2 — Constraints, Relationships, and Referential Integrity

CHECK, UNIQUE, DEFAULT • Primary & Foreign Keys • ON DELETE RESTRICT / CASCADE •
Dropping related tables

Section 1: Introduction to Constraints

Day 2 Learning Objectives

By the end of Day 2, participants should be able to:

- Explain what a constraint is and why databases use them.
- Apply CHECK constraints to validate column values.
- Apply UNIQUE constraints to prevent duplicate values.
- Apply DEFAULT values for columns that are not supplied.
- Build relationships between tables using PRIMARY KEY and FOREIGN KEY.
- Explain why PostgreSQL blocks deletes and drops that break referential integrity.
- Choose between ON DELETE RESTRICT and ON DELETE CASCADE.
- Safely drop tables that depend on one another.

What Are Constraints?

A constraint is a rule attached to a column or table that PostgreSQL enforces automatically. If a new or changed row breaks the rule, PostgreSQL rejects it and the table's data never becomes invalid.

Main constraint types

Constraint	Purpose
NOT NULL	A value must always be provided
UNIQUE	No two rows may share the same value
PRIMARY KEY	Uniquely identifies each row (UNIQUE + NOT NULL)
FOREIGN KEY	A value must exist in another table's column
CHECK	A value must satisfy a custom condition
DEFAULT	A fallback value used when none is supplied
 Constraints are what turn a plain table into a reliable source of truth: PostgreSQL enforces the rule at insert or update time, so invalid rows never reach the table.	

Section 2: The CHECK Constraint

What Is CHECK?

A CHECK constraint tests every new or updated value against a condition. If the condition is false, PostgreSQL rejects the row.

Creating a Table with a CHECK Constraint

```
SQL
CREATE TABLE products (
  product_id integer,
  name       text,
  price      numeric CHECK (price > 500)
);
```

Only rows where price is greater than 500 will be accepted.

Testing the Constraint

This insert fails because 400 is not greater than 500:

```
SQL
INSERT INTO products VALUES (1, 'oil', 400);

ERROR:  new row for relation "products" violates check constraint "products_price_check"
DETAIL:  Failing row contains (1, oil, 400).
```

The rejected row is never stored. Confirming with SELECT shows the table is still empty:

```
SQL
SELECT * FROM products;
-- (0 rows)
```

This insert succeeds because 600 satisfies the condition:

```
SQL
INSERT INTO products VALUES (1, 'oil', 600);
-- INSERT 0 1

SELECT * FROM products;
-- product_id | name | price
-- +-----+-----+-----
--          1 | oil  |  600
```

! Important: when a row violates a CHECK constraint, PostgreSQL discards the entire row — it does not partially insert it.

Adding a CHECK Constraint to an Existing Table

CHECK constraints can also be added to a column after the table already exists, using ALTER TABLE:

```
SQL
ALTER TABLE products
ADD COLUMN discount_price numeric CHECK (discount_price > 0);
```

```
SQL
```

```
SELECT * FROM products;
-- product_id | name | price | discount_price
-- -----+-----+-----+-----
--          1 | oil  |  600 |
```

Testing a negative discount price, which is rejected:

```
SQL
```

```
INSERT INTO products VALUES (2, 'soap', 700, -10);
```

```
ERROR:  new row for relation "products" violates check constraint
"products_discount_price_check"
DETAIL:  Failing row contains (2, soap, 700, -10).
```

💡 Recommendation: a table can have several CHECK constraints, one per column or even combining several columns in a single condition.

Section 3: The UNIQUE Constraint

What Is UNIQUE?

A UNIQUE constraint guarantees that no two rows share the same value in that column.

Creating a Table with UNIQUE Columns

```
SQL
```

```
CREATE TABLE courses (
  course_id integer UNIQUE,
  course_title text UNIQUE,
  lecture_no numeric
);
```

Both `course_id` and `course_title` must be unique, independently of each other.

Testing Duplicate Values

```
SQL
```

```
INSERT INTO courses VALUES (1, 'python', 24);
-- INSERT 0 1
```

Inserting the same `course_id` again is rejected:

```
SQL
```

```
INSERT INTO courses VALUES (1, 'python', 50);
```

```
ERROR:  duplicate key value violates unique constraint "courses_course_id_key"
DETAIL:  Key (course_id)=(1) already exists.
```

Inserting the same `course_title` with a different ID is also rejected:

```
SQL
```

```
INSERT INTO courses VALUES (2, 'python', 50);
```

```
ERROR: duplicate key value violates unique constraint "courses_course_title_key"
DETAIL: Key (course_title)=(python) already exists.
```

A genuinely new course_id and course_title succeed:

```
SQL
```

```
INSERT INTO courses VALUES (2, 'js', 50);
-- INSERT 0 1
```

```
SELECT * FROM courses;
-- course_id | course_title | lecture_no
-- -----+-----+-----
--          1 | python      |          24
--          2 | js         |          50
```

UNIQUE versus PRIMARY KEY

Aspect	UNIQUE	PRIMARY KEY
Duplicate values allowed?	No	No
NULL values allowed?	Yes (multiple NULLs permitted)	No
How many per table?	As many as needed	Exactly one
Typical use	Secondary identifiers (e.g. email, course title)	The main row identifier

Section 4: The DEFAULT Constraint

What Is DEFAULT?

A DEFAULT value is used automatically when an INSERT statement does not supply a value for that column.

Creating a Table with a Default Value

```
SQL
```

```
CREATE TABLE students (
  std_id integer NOT NULL,
  std_name text NOT NULL,
  address text DEFAULT ('egypt')
);
```

Inserting Without the Defaulted Column

```
SQL
```

```
INSERT INTO students VALUES (1, 'mohamed');
```

Because address was not supplied, PostgreSQL fills it in automatically:

```
SQL
SELECT * FROM students;
-- std_id | std_name | address
-- -----+-----+-----
--      1 | mohamed  | egypt
```

DEFAULT Does Not Override NOT NULL

std_name has no default, and it is NOT NULL, so a missing or NULL name is still rejected:

```
SQL
INSERT INTO students VALUES (2, NULL, 'qatar');

ERROR:  null value in column "std_name" of relation "students" violates not-null constraint
DETAIL:  Failing row contains (2, null, qatar).
```

Supplying a name succeeds and overrides nothing else:

```
SQL
INSERT INTO students VALUES (2, 'ali', 'qatar');
-- INSERT 0 1
```

Omitting address again falls back to the default:

```
SQL
INSERT INTO students VALUES (3, 'sara');
-- INSERT 0 1

SELECT * FROM students;
-- std_id | std_name | address
-- -----+-----+-----
--      1 | mohamed  | egypt
--      2 | ali      | qatar
--      3 | sara     | egypt
```

Section 5: Primary Key and Foreign Key

Recap: What Is a Primary Key?

A primary key is a column, or combination of columns, that uniquely identifies every row in a table. It cannot be NULL and cannot repeat.

What Is a Foreign Key?

A foreign key is a column whose values must exist in another table's referenced column (usually that table's primary key). It is how PostgreSQL enforces relationships between tables.

Creating Related Tables

SQL

```
CREATE TABLE products (
  product_id integer PRIMARY KEY,
  name       text,
  price      numeric
);
```

SQL

```
CREATE TABLE orders (
  order_id   integer PRIMARY KEY,
  product_id integer REFERENCES products(product_id),
  quantity   numeric
);
```

⚠ Important correction: the column referencing products should be named `product_id`, matching the column it points to. A misspelled name such as `prodcut_id` still works technically, but it is confusing and error-prone — keep foreign key column names consistent with the table they reference.

Inserting Valid Related Data

SQL

```
INSERT INTO products VALUES (1, 'tea', 10);
INSERT INTO products VALUES (2, 'coffee', 9);

INSERT INTO orders VALUES (1, 1, 6);
INSERT INTO orders VALUES (2, 2, 7);
```

Both orders reference product IDs that already exist in products, so both inserts succeed.

Foreign Key Enforcement on Insert

Inserting an order for a `product_id` that does not exist in products is rejected:

SQL

```
INSERT INTO orders VALUES (3, 3, 1);

ERROR: insert or update on table "orders" violates foreign key constraint
"orders_product_id_fkey"
DETAIL: Key (product_id)=(3) is not present in table "products".
```



This is the core benefit of a foreign key: it is impossible to create an order for a product that does not exist.

Section 6: Referential Integrity and Deletion

Why Can't We Delete a Referenced Row?

Deleting a product that an order still points to is rejected by default:

SQL

```
DELETE FROM products WHERE product_id = 1;
```

```
ERROR: update or delete on table "products" violates foreign key constraint
"orders_product_id_fkey" on table "orders"
DETAIL: Key (product_id)=(1) is still referenced from table "orders".
```

✂ PostgreSQL protects the orders table from ending up with an order that points to a product that no longer exists.

Why Can't We Drop a Referenced Table?

```
SQL
```

```
DROP TABLE products;
```

```
ERROR: cannot drop table products because other objects depend on it
DETAIL: constraint orders_product_id_fkey on table orders depends on table products
HINT: Use DROP ... CASCADE to drop the dependent objects too.
```

⚠ *Correction: the safe approach is to drop the dependent table first, then the referenced table — not to reach for DROP ... CASCADE by default, since CASCADE would also remove the constraint (and can remove dependent objects) without warning.*

```
SQL
```

```
DROP TABLE orders;
-- DROP TABLE

DROP TABLE products;
-- DROP TABLE
```

Section 7: ON DELETE Actions — RESTRICT and CASCADE

Why Choose a Delete Action?

A foreign key can declare what should happen to child rows when the referenced parent row is deleted. This is set with ON DELETE when the foreign key is created.

ON DELETE Option	Behavior
RESTRICT (or the default)	Blocks the delete if any row still references the parent
CASCADE	Automatically deletes the matching child rows too
SET NULL	Sets the child's foreign key column to NULL
NO ACTION	Similar to RESTRICT, checked at the end of the statement

ON DELETE RESTRICT

Setup: an items table and an orders table that explicitly restricts deletion.

```
SQL
```

```
CREATE TABLE items (
  item_id integer PRIMARY KEY,
  name    text
);
```

```
SQL
```

```
CREATE TABLE orders (
  order_id integer PRIMARY KEY,
  item_id  integer REFERENCES items(item_id) ON DELETE RESTRICT,
  quantity numeric
);
```

Attempting to delete a referenced item is blocked, just like the default behavior:

```
SQL
```

```
DELETE FROM items WHERE item_id = 1;
```

```
ERROR: update or delete on table "items" violates RESTRICT setting of foreign key
constraint "orders_item_id_fkey" on table "orders"
DETAIL: Key (item_id)=(1) is referenced from table "orders".
```

ON DELETE CASCADE

To allow deleting a parent row and automatically remove its dependent child rows, drop and recreate the foreign key with CASCADE:

```
SQL
```

```
DROP TABLE orders;
```

```
CREATE TABLE orders (
  order_id integer PRIMARY KEY,
  item_id  integer REFERENCES items(item_id) ON DELETE CASCADE,
  quantity numeric
);
```

Now deleting an item also removes any orders that referenced it, instead of being blocked:

```
SQL
```

```
DELETE FROM items WHERE item_id = 1;
-- DELETE 1
-- Query returned successfully.
```

⚠ Warning: ON DELETE CASCADE is powerful but destructive — deleting one parent row silently deletes every dependent child row. Use it only when that cascading deletion is genuinely the desired behavior.

Choosing Between RESTRICT and CASCADE

- Use RESTRICT (or leave the default) when child rows should never be orphaned silently — e.g. products with existing orders.
- Use CASCADE when child rows have no meaning without the parent — e.g. deleting a course should remove its enrollments.

- Always confirm with a SELECT on the child table before deleting a parent row, especially when CASCADE is in effect.

Section 8: Day 2 Review

What We Learned

- What a constraint is and why PostgreSQL enforces it automatically.
- How CHECK validates values against a custom condition.
- How UNIQUE prevents duplicate values, and how it differs from PRIMARY KEY.
- How DEFAULT supplies a fallback value when none is given.
- How FOREIGN KEY enforces that child values exist in the parent table.
- Why PostgreSQL blocks deletes and drops that would break referential integrity.
- The difference between ON DELETE RESTRICT and ON DELETE CASCADE.

Command Summary

```
SQL
-- CHECK
price numeric CHECK (price > 500)

-- UNIQUE
course_id integer UNIQUE

-- DEFAULT
address text DEFAULT ('egypt')

-- PRIMARY KEY / FOREIGN KEY
product_id integer PRIMARY KEY
product_id integer REFERENCES products(product_id)

-- ON DELETE actions
item_id integer REFERENCES items(item_id) ON DELETE RESTRICT
item_id integer REFERENCES items(item_id) ON DELETE CASCADE
```

Knowledge Check

1. What happens to a row that violates a CHECK constraint?
2. Can a UNIQUE column contain more than one NULL value?
3. What is the main difference between UNIQUE and PRIMARY KEY?
4. What value does PostgreSQL use when an INSERT omits a column with a DEFAULT?
5. Why does PostgreSQL reject an order that references a non-existent product?
6. Why can't you drop a table that another table depends on?
7. What is the difference between ON DELETE RESTRICT and ON DELETE CASCADE?

Day 2 Practical Laboratory

Laboratory Objectives

During the lab, participants will:

- Add CHECK constraints to validate numeric values.
- Add UNIQUE constraints to prevent duplicate values.
- Add DEFAULT values for optional columns.
- Build a PRIMARY KEY / FOREIGN KEY relationship between two tables.
- Test referential integrity on insert, delete, and drop.
- Compare ON DELETE RESTRICT and ON DELETE CASCADE.

Lab Scenario

You are building a small store database with products and the orders placed for them.

Business rules

- A product's price must always be greater than 0.
- A product's name must be unique.
- A new order defaults to a quantity of 1 if none is given.
- Every order must reference a product that actually exists.
- Deleting a product that still has orders should be prevented, unless the lab explicitly asks to test CASCADE.

Lab Part A: CHECK and UNIQUE

```
SQL
CREATE TABLE products (
  product_id integer PRIMARY KEY,
  name       text UNIQUE,
  price      numeric CHECK (price > 0)
);
```

Test: insert a product with a negative price and confirm it is rejected. Insert two products with the same name and confirm the second is rejected.

Lab Part B: DEFAULT

```
SQL
CREATE TABLE orders (
  order_id   integer PRIMARY KEY,
  product_id integer REFERENCES products(product_id),
  quantity   numeric DEFAULT 1
);
```

Test: insert an order without a quantity and confirm it defaults to 1.

Lab Part C: Foreign Key Enforcement

Insert two valid products, then two valid orders that reference them. Then attempt to insert an order that references a `product_id` that does not exist, and record the error message.

Lab Part D: Referential Integrity on Delete

Attempt to delete a product that has an existing order. Record the error message, then delete the order first, then the product.

Lab Part E: RESTRICT versus CASCADE

Recreate the orders table so that `product_id` uses ON DELETE CASCADE. Insert a product and an order referencing it, then delete the product and confirm the order disappears automatically.

```
SQL

DROP TABLE orders;

CREATE TABLE orders (
  order_id integer PRIMARY KEY,
  product_id integer REFERENCES products(product_id) ON DELETE CASCADE,
  quantity numeric DEFAULT 1
);
```

Lab Safety Exercise

? Before deleting any product, first run a SELECT to check whether any orders reference it.

```
SQL

SELECT * FROM orders WHERE product_id = 1;
```

 Expected answer: checking first shows exactly which orders would be affected — critical when the foreign key uses ON DELETE CASCADE, since those rows would be silently removed.

Practical Challenge

Extend the schema with a categories table.

Requirements

- `category_id` as a primary key.
- `category_name` that is required and unique.
- Add a `category_id` column to products that references `categories(category_id)` with ON DELETE RESTRICT.
- Insert at least three categories and assign them to the existing products.
- Attempt to delete a category that is still in use, and confirm it is blocked.

Lab Submission Requirements

8. One .sql script containing all commands.
9. Screenshots of each rejected insert/delete with its error message.
10. Results of the final SELECT statements for products, orders, and categories.
11. A short written answer to the lab safety question.

Day 2 Lab Evaluation

Criterion	Weight
CHECK and UNIQUE constraints applied correctly	20%
DEFAULT value behaves as expected	10%
Primary key / foreign key relationship built correctly	20%
Referential integrity correctly tested and explained	20%
RESTRICT vs CASCADE correctly demonstrated	20%
Script organization and comments	10%
Total	100%

codes

Part 1: CHECK Constraint

```

SQL

CREATE TABLE products (
  product_id integer,
  name       text,
  price      numeric CHECK (price > 500)
);

INSERT INTO products VALUES (1, 'oil', 400); -- rejected
INSERT INTO products VALUES (1, 'oil', 600); -- accepted

ALTER TABLE products
ADD COLUMN discount_price numeric CHECK (discount_price > 0);

INSERT INTO products VALUES (2, 'soap', 700, -10); -- rejected

```

Part 2: UNIQUE Constraint

```

SQL

CREATE TABLE courses (
  course_id   integer UNIQUE,
  course_title text    UNIQUE,
  lecture_no  numeric

```

```
);

INSERT INTO courses VALUES (1, 'python', 24);
INSERT INTO courses VALUES (1, 'python', 50); -- rejected: duplicate course_id
INSERT INTO courses VALUES (2, 'python', 50); -- rejected: duplicate course_title
INSERT INTO courses VALUES (2, 'js', 50);      -- accepted
```

Part 3: DEFAULT Constraint

```
SQL

CREATE TABLE students (
  std_id   integer NOT NULL,
  std_name text    NOT NULL,
  address  text    DEFAULT ('egypt')
);

INSERT INTO students VALUES (1, 'mohamed');
INSERT INTO students VALUES (2, NULL, 'qatar'); -- rejected: NOT NULL
INSERT INTO students VALUES (2, 'ali', 'qatar');
INSERT INTO students VALUES (3, 'sara');        -- address defaults to 'egypt'
```

Part 4: Primary Key and Foreign Key

```
SQL

CREATE TABLE products (
  product_id integer PRIMARY KEY,
  name       text,
  price      numeric
);

CREATE TABLE orders (
  order_id   integer PRIMARY KEY,
  product_id integer REFERENCES products(product_id),
  quantity   numeric
);

INSERT INTO products VALUES (1, 'tea', 10);
INSERT INTO products VALUES (2, 'coffee', 9);
INSERT INTO orders VALUES (1, 1, 6);
INSERT INTO orders VALUES (2, 2, 7);
INSERT INTO orders VALUES (3, 3, 1); -- rejected: no such product
```

Part 5: Referential Integrity

```
SQL

DELETE FROM products WHERE product_id = 1; -- rejected: referenced by orders
DROP TABLE products;                     -- rejected: orders depends on it

DROP TABLE orders;                       -- succeeds
DROP TABLE products;                     -- now succeeds
```

Part 6: ON DELETE RESTRICT and CASCADE

SQL

```
CREATE TABLE items (  
    item_id integer PRIMARY KEY,  
    name    text  
);  
  
CREATE TABLE orders (  
    order_id integer PRIMARY KEY,  
    item_id  integer REFERENCES items(item_id) ON DELETE RESTRICT,  
    quantity numeric  
);  
  
DELETE FROM items WHERE item_id = 1; -- rejected: RESTRICT  
  
DROP TABLE orders;  
  
CREATE TABLE orders (  
    order_id integer PRIMARY KEY,  
    item_id  integer REFERENCES items(item_id) ON DELETE CASCADE,  
    quantity numeric  
);  
  
DELETE FROM items WHERE item_id = 1; -- succeeds, matching orders removed too
```